Привет я реализуя состенную pytorch подобную библиотеку длоя c# использующую opencl и уменя возникла интересная идея.
Сейчас архитектуру бибилотеки можно описать следующим образом:

```
gate_A, gate_B, reader_C = nnclglob.split(projected, [self.d_state, self.d_state, self.d_model], dim=-1)

nnclglob. LocalRunner(()=>
        get_A=nncllоc.sigmoid(get_A)
        get_B=nncllоc.tanh(get_B)

)
new_information = nnclglob.bmm(v, gate_B)
nnclglob. LocalRunner(()=>
        memory_state=memory_state*get_A
        memory_state=memory_state+new_information
)
tmp=nnclglob.transpose( memory_state,1, 2)
current_output=nnclglob.bmm(tmp,reader_C)
```

Решение проблемы обратного проохода

Чтобы решить проблему обратиного прохода, можно параллельно с основной цепочкой (при включенном реждиме обучения) строить 2 цепочку обратного прохода , заменяя функции их производными.

```
    Ссылок: 0
    public Tensor Forward(Tensor x)
    {
        // 1. Слой нормализации (теперь RmsGamma гарантированно состоит из 1.0f)
        Tensor normalized = NnClGlob.RmsNorm(x, RmsGamma);

        // 2. Линейная проекция через пакетное умножение матриц
        Tensor projected = NnClGlob.Bmm(normalized, ProjectionWeights);

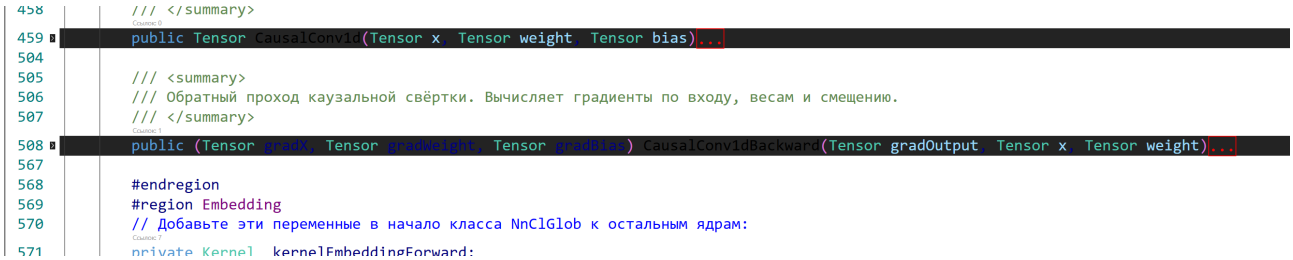
        // ИНТЕГРАЦИЯ: Безопасно скачиваем чистые данные проекции на хост (CPU)
        ProjectedHost = HostReturner.ToCpu(projected);

        // 3. Вычисление вероятностей распределения внимания
        Tensor probabilities = NnClGlob.Softmax(projected, dim: -1);

        return probabilities;
    }
}
```

```
92      // ========================================================================
93      // РЕАЛИЗАЦИЯ ОПЕРАЦИЙ (FORWARD PASS)
94      // ========================================================================
95
96      /// <summary>
97      /// Глобальная безопасная операция Softmax (Forward).
98      /// Рассчитана на работу со скорами внимания размерности [B * H * L, SEQ_LEN].
99      /// </summary>
        Ссылок: 2
100     public Tensor Softmax(Tensor x, int dim = -1)...
143     /// <summary>
144     /// Обратный проход глобальной операции Softmax.
145     /// Использует уже вычисленный forward-выход тензора Y для экономии памяти VRAM.
146     /// </summary>
147     /// <param name="gradOutput">Пришедший сверху градиент dL/dY [B * H * L, SEQ_LEN]</param>
148     /// <param name="y">Исходный выход прямого прохода Softmax (вероятности) [B * H * L, SEQ_LEN]</param>
149     /// <returns>Выходной градиент по входу dL/dX [B * H * L, SEQ_LEN]</returns>
        Ссылок: 1
150     public Tensor SoftmaxBackward(Tensor gradOutput, Tensor y)...
177     #region RSNORM
178     // Добавьте эти переменные в начало класса NnClGlob к остальным ядрам:
        Ссылок: 7
179     private Kernel _kernelRmsNormForward;
        Ссылок: 9
180     private Kernel _kernelRmsNormBackward;
181
182     // Добавьте их инициализацию в метод LoadKernels():
183     // _kernelRmsNormForward = _prgRmsNorm.CreateKernel("rmsnorm_forward");
184     // _kernelRmsNormBackward = _prgRmsNorm.CreateKernel("rmsnorm_backward");
185
186     /// <summary>
187     /// Глобальная операция RMSNorm (Forward).
188     /// Обрабатывает тензор любой формы, выполняя нормализацию по последней оси d_model.
189     /// </summary>
        Ссылок: 2
190     public Tensor RmsNorm(Tensor x, Tensor gamma)...
237
238     /// <summary>
239     /// Обратный проход глобальной операции RMSNorm.
240     /// Вычисляет градиенты по входу X и по весам Gamma
```

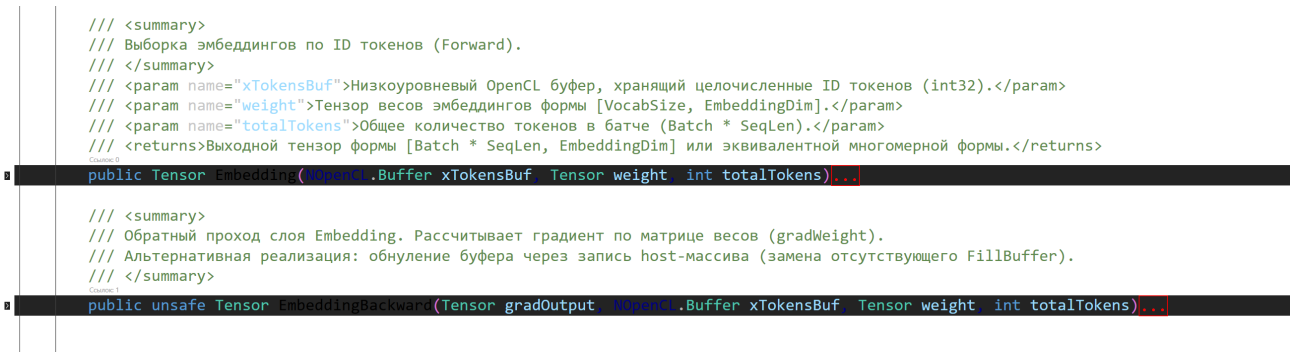
```
314     /// </summary>
        Ссылок: 2
315     public unsafe Tensor Permute(Tensor x, int[] axes)...
357
358     #endregion
359     #region BMM
360     // Добавьте эту переменную в начало класса NnClGlob к остальным ядрам:
        Ссылок: 11
361     private Kernel _kernelBmmTiled;
362
363     // Добавьте её инициализацию в метод LoadKernels():
364     // _kernelBmmTiled = _prgMatMul.CreateKernel("bmm_tiled");
365
366     /// <summary>
367     /// Пакетное матричное умножение (Batch Matrix Multiplication).
368     /// Ожидает строго 3D-тензоры: A [Batch, M, K], B [Batch, K, N] -> Выход [Batch, M, N].
369     /// </summary>
370     /// <summary>
371     /// Пакетное матричное умножение с поддержкой Batch Broadcasting для весов модели.
372     /// </summary>
        Ссылок: 5
373     public Tensor Bmm(Tensor a, Tensor b)...
439
440
441     #endregion
```



```
        /// </summary>
        public Tensor CausalConv1d(Tensor x, Tensor weight, Tensor bias)...

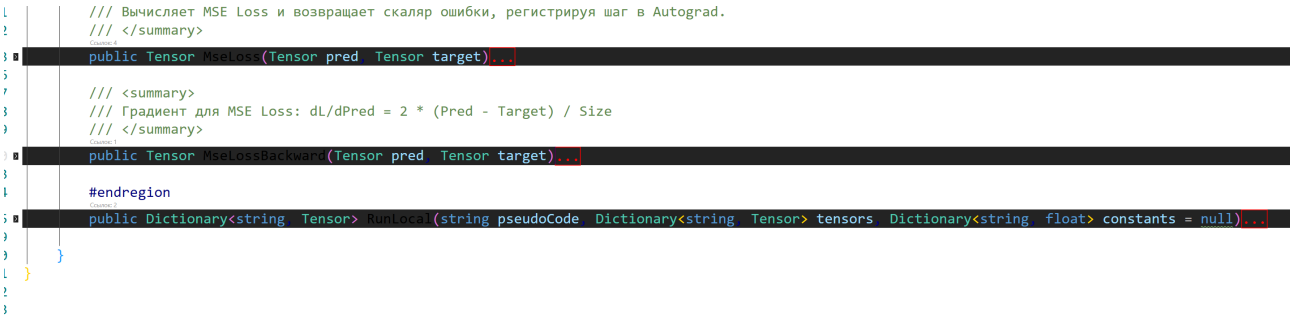
        /// <summary>
        /// Обратный проход каузальной свёртки. Вычисляет градиенты по входу, весам и смещению.
        /// </summary>
        public (Tensor gradX, Tensor gradWeight, Tensor gradBias) CausalConv1dBackward(Tensor gradOutput, Tensor x, Tensor weight)...

        #endregion
        #region Embedding
        // Добавьте эти переменные в начало класса NnClGlob к остальным ядрам:
        private Kernel kernelEmbeddingForward;
```



```
    /// <summary>
    /// Выборка эмбеддингов по ID токенов (Forward).
    /// </summary>
    /// <param name="xTokensBuf">Низкоуровневый OpenCL буфер, хранящий целочисленные ID токенов (int32).</param>
    /// <param name="weight">Тензор весов эмбеддингов формы [VocabSize, EmbeddingDim].</param>
    /// <param name="totalTokens">Общее количество токенов в батче (Batch * SeqLen).</param>
    /// <returns>Выходной тензор формы [Batch * SeqLen, EmbeddingDim] или эквивалентной многомерной формы.</returns>
    public Tensor Embedding(NOpenCL.Buffer xTokensBuf, Tensor weight, int totalTokens)...

    /// <summary>
    /// Обратный проход слоя Embedding. Рассчитывает градиент по матрице весов (gradWeight).
    /// Альтернативная реализация: обнуление буфера через запись host-массива (замена отсутствующего FillBuffer).
    /// </summary>
    public unsafe Tensor EmbeddingBackward(Tensor gradOutput, NOpenCL.Buffer xTokensBuf, Tensor weight, int totalTokens)...
```



```
    /// Вычисляет MSE Loss и возвращает скаляр ошибки, регистрируя шаг в Autograd.
    /// </summary>
    public Tensor MseLoss(Tensor pred, Tensor target)...

    /// <summary>
    /// Градиент для MSE Loss: dL/dPred = 2 * (Pred - Target) / Size
    /// </summary>
    public Tensor MseLossBackward(Tensor pred, Tensor target)...

    #endregion
    public Dictionary<string, Tensor> RunLocal(string pseudoCode, Dictionary<string, Tensor> tensors, Dictionary<string, float> constants = null)...
}
}
```

```
    public class Linear : NeuralNetwork
    {
        /// <summary>
        /// Обучаемая матрица весов слоя. Форма: [1, InFeatures, OutFeatures]
        /// </summary>
        public Tensor Weight { get; private set; }

        /// <summary>
        /// Обучаемый вектор смещения (bias). Форма: [1, OutFeatures] — будет
автоматически растягиваться по батчу.
        /// </summary>
        public Tensor Bias { get; private set; }

        private readonly int _inFeatures;
        private readonly int _outFeatures;

        /// <summary>
```

```
        /// Инициализирует линейный слой и заполняет его веса случайным шумом
для разрушения симметрии.
        /// </summary>
        public Linear(ComputeContext computeContext, NnClGlob nnClGlob, int
inFeatures, int outFeatures)
            : base(computeContext, nnClGlob)
        {
            _inFeatures = inFeatures;
            _outFeatures = outFeatures;

            // 1. Аллокация памяти во VRAM девайса
            // Веса делаем 3D формы [1, K, N] под требования нашего пакетного
ядра BMM
            Weight = RegisterParameter(new[] { 1, inFeatures, outFeatures },
requiresGrad: true);
            // Bias делаем 2D формы [1, OutFeatures] под требования Broadcasting
            Bias = RegisterParameter(new[] { 1, outFeatures }, requiresGrad:
true);

            // 2. Инициализация весов методом Ксавье (Xavier/Glorot
Initialization)
            // Это стандарт глубокого обучения, предохраняющий tanh от насыщения
и зануления градиентов
            CommandQueue queue = DeviceQueueRegistry.GetCurrent();
            var rand = new Random();

            float bound = (float)(Math.Sqrt(6.0) / Math.Sqrt(inFeatures +
outFeatures));

            // Заполнение матрицы весов W
            float[] hostWeights = new float[Weight.Size];
            for (int i = 0; i < hostWeights.Length; i++)
            {
                hostWeights[i] = (float)(rand.NextDouble() * 2.0 - 1.0) * bound;
            }
            long weightsBytes = (long)Weight.Size * sizeof(float);
            unsafe
            {
                fixed (float* hw = hostWeights)
                {
 queue.EnqueueWriteBuffer(Weight.DataBuffer, true, 0, weightsBytes, (IntPtr)hw);
                }
            }


            // Заполнение вектора смещения Bias (по умолчанию заполняем нулями,
как в PyTorch)
            float[] hostBias = new float[Bias.Size];
            long biasBytes = (long)Bias.Size * sizeof(float);
            unsafe
            {
                fixed (float* hb = hostBias)
                {
queue.EnqueueWriteBuffer(Bias.DataBuffer, true, 0, biasBytes, (IntPtr)hb);
                }
            }


            queue.Finish();
        }

        /// <summary>
        /// Выполняет высокооптимизированный прямой проход: BMM -> JIT Сложение
с Bias.
```

```
        /// На вход ожидает 3D тензор данных формы [Batch, M, InFeatures].
        /// </summary>
        public Tensor Forward(Tensor x)
        {
            if (x.Shape[x.Shape.Length - 1] != _inFeatures)
            {
                throw new ArgumentException($"Несоответствие размерностей входа.
Ожидалось {_inFeatures} признаков, получено {x.Shape[x.Shape.Length - 1]}.");
            }

            // ШАГ 1 (ГЛОБАЛЬНЫЙ): Тяжелое матричное умножение X x W на GPU
            // Выход projected получит форму [Batch, M, OutFeatures]
            Tensor projected = NnClGlob.Bmm(x, Weight);

            // ШАГ 2 (ЛОКАЛЬНЫЙ JIT): Сплавление операции прибавления Bias на
регистрах GPU.
            // Передаем projected и bias в наш Roslyn-компонент
            var tensorArgs = new Dictionary<string, Tensor> {
                { "P", projected },
                { "b", Bias }
            };

            // Математический псевдокод для JIT.
            // Благодаря нашему модулю Broadcasting, вектор 'b' автоматически
прибавится к каждой строке матрицы 'P'!
            string pseudoCode = "P = P + b;";

            // Вызываем RunLocal, который вернет НОВЫЙ чистый тензор без
затирания графа Autograd
            Dictionary<string, Tensor> localOutputs =
NnClGlob.RunLocal(pseudoCode, tensorArgs, constants: null);
            Tensor output = localOutputs["P"];

            return output;
        }
    }
```

```
 /// <summary>
 /// Глобальная безопасная операция Softmax (Forward).
 /// Рассчитана на работу со скорами внимания размерности [B * H * L, SEQ_LEN].
 /// </summary>
 public Tensor Softmax(Tensor x, int dim = -1)
 {
     // Эквивалент проверки размерности в Python: assert dim == -1 or dim ==
len(x.shape) - 1
     int targetDim = dim == -1 ? x.Shape.Length - 1 : dim;
     if (targetDim != x.Shape.Length - 1)
     {
         throw new NotSupportedException("Текущая реализация ядра Softmax
поддерживает редукцию только по последней оси.");
     }

     int seqLen = x.Shape[x.Shape.Length - 1]; // Последняя ось
     int numRows = x.Size / seqLen;            // Вычисляем количество строк

     // Создаем выходной тензор той же формы во VRAM (RequiresGrad пока передаем
как у исходного)
     Tensor y = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: x.RequiresGrad);
```

```
        // Настройка сетки потоков (Сетка идентична вашей конфигурации на Python)
        IntPtr localSize = new IntPtr(256);
        IntPtr globalSize = new IntPtr(numRows * 256);

        // Получаем глобальную активную очередь выполнения задач из нашего реестра
        CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();

        // Передаем аргументы в Си-ядро по порядку индексов (0, 1, 2, 3):
        // kernel void softmax_forward(global const float* X, global float* Y,
const int seq_len)
        _kernelSoftmaxForward.Arguments[0].SetValue(x.DataBuffer);
        _kernelSoftmaxForward.Arguments[1].SetValue(y.DataBuffer);
        _kernelSoftmaxForward.Arguments[2].SetValue(seqLen);

        // Запуск выполнения ядра на GPU
        activeQueue.EnqueueNDRangeKernel(_kernelSoftmaxForward, null, new[] {
globalSize }, new[] { localSize });

        // Синхронизация: гарантирует завершение вычислений перед чтением
        activeQueue.Finish();

        // Если включен режим Autograd и требуется расчет градиента, привязываем
контекст создателя к тензору
        if (y.RequiresGrad)
        {
            // Создаем операцию, передавая входы и выходы
            y.Creator = new SoftmaxOp(x, y, this);
        }

        return y;
    }
    /// <summary>
    /// Обратный проход глобальной операции Softmax.
    /// Использует уже вычисленный forward-выход тензора Y для экономии памяти
VRAM.
    /// </summary>
    /// <param name="gradOutput">Пришедший сверху градиент dL/dY [B * H * L,
SEQ_LEN]</param>
    /// <param name="y">Исходный выход прямого прохода Softmax (вероятности) [B * H
* L, SEQ_LEN]</param>
    /// <returns>Выходной градиент по входу dL/dX [B * H * L, SEQ_LEN]</returns>
    public Tensor SoftmaxBackward(Tensor gradOutput, Tensor y)
    {
        int seqLen = y.Shape[y.Shape.Length - 1];
        int numRows = y.Size / seqLen;

        // Выделяем тензор для результирующего градиента по входу (по форме
совпадает с Y)
        Tensor gradInput = new Tensor(y.Shape, _computeCtx.Context,
MemoryFlags.ReadWrite, requiresGrad: false);

        // Настройка сетки потоков (идентично forward шагу)
        IntPtr localSize = new IntPtr(256);
        IntPtr globalSize = new IntPtr(numRows * 256);

        CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
        int t = 0;
        // Передаем аргументы в Си-ядро softmax_backward:
        // kernel void softmax_backward(global const float* grad_output, global
const float* Y, global float* grad_input, const int seq_len)
        _kernelSoftmaxBackward.Arguments[t++].SetValue(gradOutput.DataBuffer);
        _kernelSoftmaxBackward.Arguments[t++].SetValue(y.DataBuffer);
        _kernelSoftmaxBackward.Arguments[t++].SetValue(gradInput.DataBuffer);
        _kernelSoftmaxBackward.Arguments[t++].SetValue(seqLen);
```

```
        // Запуск ядра
        activeQueue.EnqueueNDRangeKernel(_kernelSoftmaxBackward, null, new[] {
globalSize }, new[] { localSize });
        activeQueue.Finish();

        return gradInput;
    }
```

```
Эпоха 18 | Текущий MSE Loss: 1,620434
============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [1,1272, 0,6283, 0,9345, 1,7357, -0,5058, -0,0291, 0,3140, 1,0219]

Градиенты (Backward):
  [0,0318, -0,3429, -0,5164, -0,5661, 0,1235, -0,0073, -0,1715, -0,2445]
============================================================


Эпоха 19 | Текущий MSE Loss: 1,532150
============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [1,1339, 0,6742, 0,9817, 1,7831, -0,5471, -0,0541, 0,3570, 1,0665]

Градиенты (Backward):
  [0,0335, -0,3315, -0,5046, -0,5542, 0,1132, -0,0135, -0,1608, -0,2334]
============================================================


Эпоха 20 | Текущий MSE Loss: 1,447747
============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 2, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [1,1365, 0,7196, 1,0284, 1,8301, -0,5873, -0,0761, 0,3991, 1,1105]

Градиенты (Backward):
  [0,0341, -0,3201, -0,4929, -0,5425, 0,1032, -0,0190, -0,1502, -0,2224]
============================================================


------------------------------------------
Цикл завершен. Проверяем финальные веса на GPU...

Сравнение результатов (Target vs Trained):
Элемент [0]: Идеал = 1,00 | Обучено = 1,1356
Элемент [1]: Идеал = 2,00 | Обучено = 0,7645
Элемент [2]: Идеал = 3,00 | Обучено = 1,0749
Элемент [3]: Идеал = 4,00 | Обучено = 1,8768
Элемент [4]: Идеал = -1,00 | Обучено = -0,6264
Элемент [5]: Идеал = 0,00 | Обучено = -0,0948
Элемент [6]: Идеал = 1,00 | Обучено = 0,4403
Элемент [7]: Идеал = 2,00 | Обучено = 1,1537

=== ТЕСТ ЗАВЕРШЕН ===

C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\csharp\cstorchcl\AdamTest\bin\Debug\net10.0\AdamTest.exe (процесс 16076) завершил работ
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыт
ке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [2, 1, 4]
. Всего элементов: 8
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [-0,0295, -3,2246, 2,2725, -0,5703, -0,8542, -7,6978, 5,3771, -2,9924]

Градиенты (Backward):
  [0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000]
============================================================


============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 4]
. Всего элементов: 4
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [0,0000, 0,3698, 0,0000, 0,0000]

Градиенты (Backward):
  [0,0000, -0,1576, 0,0000, 0,0000]
============================================================


============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [2, 1, 3]
. Всего элементов: 6
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [1,0000, 2,0000, 3,0000, 4,0000, 5,0000, 6,0000]

Градиенты (Backward):
  [1,0000, 2,0000, 3,0000, 4,0000, 5,0000, 6,0000]
============================================================


============================================================
ОТЛАДКА ТЕНЗОРА: sum_grad
. Форма (Shape): [1, 3, 4]
. Всего элементов: 12
. Тип данных: System.Single (float32)
============================================================
Данные (Forward):
  [-0,4909, -0,1819, 0,4360, -0,6982, 0,1866, -0,8848, -0,0399, -0,4554, 0,0294, -0,4244, 0,6387, 0,3462]

Градиенты (Backward):
  [0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000, 0,0000]
============================================================


[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [0,0000, -0,1576, 0,0000, 0,0000]
        Градиент весов Weight (первые 4): 0,00000, 0,00000, 0,00000, 0,00000

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.

=== ТЕСТ ОБУЧАЕМОГО СЛОЯ LINEAR С УСПЕХОМ ЗАВЕРШЕН! ===
```

```
namespace cstorchcl.core
{
    /// <summary>
    /// Базовый абстрактный класс для всех математических операций в графе
вычислений.
    /// Хранит контекст связей между тензорами и логику вычисления производных.
    /// </summary>
    public abstract class Op
    {
        /// <summary>
        /// Массив входных тензоров, которые участвовали в данной операции.
        /// </summary>
        public Tensor[] Inputs { get; protected set; }

        /// <summary>
        /// Выходной тензор, который был получен в результате данной операции.
        /// </summary>
        public Tensor Output { get; protected set; }

        /// <summary>
        /// Выполняет обратный проход операции: рассчитывает градиенты по входам
        /// на основе пришедшего сверху градиента по выходу.
        /// </summary>
        /// <param name="gradOutput">Градиент функции потерь по выходу этой
операции (dL/dY).</param>
        /// <param name="context">Текущий рабочий контекст OpenCL.</param>
        /// <returns>Массив градиентов по входам операции (dL/dX_i).</returns>
        public abstract Tensor[] Backward(Tensor gradOutput, Context context);
    }

    /// <summary>
    /// Статический движок автоматического дифференцирования.
    /// </summary>
    public static class AutoGrad
    {
        /// <summary>
        /// Запускает топологическую сортировку графа и выполняет обратное
распространение ошибки (Backpropagation).
        /// </summary>
        /// <param name="target">Целевой тензор (обычно это скаляр Loss, от
которого берется производная).</param>
        /// <param name="context">Текущий рабочий контекст OpenCL для аллокации
буферов градиентов.</param>
        public static void Backward(Tensor target, Context context)
        {
            if (target == null) throw new ArgumentNullException(nameof(target));
            if (context == null) throw new
ArgumentNullException(nameof(context));

            // 1. Инициализируем градиент целевого тензора (для Loss это всегда
единица: dLoss/dLoss = 1.0f)
            target.RequiresGrad = true;
            target.InitGradient(context);

            // Заполняем буфер единицей. В OpenCL это делается через очередь,
            // но для базовой логики графа считаем, что буфер
target.Grad.DataBuffer готов.
            // (В будущем вызовем activeQueue.EnqueueFillBuffer)

            // 2. Строим топологический порядок узлов графа вычислений (от
выходов к входам)
            List<Tensor> topoOrder = new List<Tensor>();
```

```
            HashSet<Tensor> visited = new HashSet<Tensor>();

            void BuildTopo(Tensor node)
            {
                if (visited.Contains(node)) return;
                visited.Add(node);

                // Если тензор был создан операцией (это не лист дерева весов), идем глубже по его входам
                if (node.Creator is Op op)
                {
                    foreach (var input in op.Inputs)
                    {
                        BuildTopo(input);
                    }
                }
                topoOrder.Add(node);
            }

            BuildTopo(target);

            // Разворачиваем список, чтобы идти строго в обратном порядке (от Loss назад к весам)
            topoOrder.Reverse();

            // 3. Выполняем обход графа и аккумулируем градиенты
            foreach (var tensor in topoOrder)
            {
                if (tensor.Creator is Op op)
                {
                    // Запрашиваем у операции расчет градиентов для ее входов
                    Tensor[] inputGrads = op.Backward(tensor.Grad, context);

                    for (int i = 0; i < op.Inputs.Length; i++)
                    {
                        Tensor input = op.Inputs[i];
                        if (!input.RequiresGrad) continue;

                        // Инициализируем буфер градиента для входа, если он еще пустой

                        input.InitGradient(context);

                        // Аккумулируем градиент (input.Grad += inputGrads[i])
                        // ВАЖНО: В нейросетях градиенты одного и того же тензора, используемого в разных местах, складываются.
                        // Для этого потребуется написать поэлементное суммирующее OpenCL-ядро (Add-kernel)
                        AccumulateGradient(input.Grad, inputGrads[i]);
#if DEBUG
HostReturner.PrintTensor(input, "sum_grad", true);
                        Console.WriteLine();
#endif

                    }
                }
            }
        }

        /// <summary>
        /// Физически суммирует градиенты на GPU во VRAM.
        /// Реализует операцию: targetGrad += newGrad
        /// </summary>
```

```
        private static void AccumulateGradient(Tensor targetGrad, Tensor
newGrad)
        {
            if (targetGrad == null || newGrad == null) return;

            string kernelSource = @"
                kernel void accum_grad(global float* target, global const float*
source) {
                    int id = get_global_id(0);
                    target[id] += source[id];
                }";

            // Используем контекст и устройство из буфера тензора
            // (Так как NOpenCL.Buffer привязан к контексту, берем глобальный
ComputeContext через диспетчер или activeQueue)
            var queue = DeviceQueueRegistry.GetCurrent();

            // Компилируем легкое ядро аккумуляции
            var prg = compiler.JITProgramCache.GetOrCompile(queue.Context,
queue.Device, kernelSource);
            using var kernel = prg.CreateKernel("accum_grad");

            kernel.Arguments[0].SetValue(targetGrad.DataBuffer);
            kernel.Arguments[1].SetValue(newGrad.DataBuffer);

            // Запускаем параллельное сложение по всей длине буфера градиента
            queue.EnqueueNDRangeKernel(kernel,  new[] { new
IntPtr(targetGrad.Size) }, null);
            queue.Finish();
        }

    }
}

namespace cstorchcl.core
{
    /// <summary>
    /// Операция графа автодифференцирования для слоя выборки эмбеддингов
(Embedding).
    /// </summary>
    public class EmbeddingOp : Op
    {
        private readonly NOpenCL.Buffer _xTokensBuf;
        private readonly int _totalTokens;
        private readonly NnClGlob _nnClGlob;

        /// <summary>
        /// Создает узел операции Embedding в графе вычислений.
        /// </summary>
        public EmbeddingOp(NOpenCL.Buffer xTokensBuf, Tensor weight, Tensor
output, int totalTokens, NnClGlob nnClGlob)
        {
            // У операции один дифференцируемый вход — матрица весов (Weight)
            Inputs = new[] { weight };
            Output = output;

            _xTokensBuf = xTokensBuf ?? throw new
ArgumentNullException(nameof(xTokensBuf));
            _totalTokens = totalTokens;
            _nnClGlob = nnClGlob ?? throw new
ArgumentNullException(nameof(nnClGlob));
        }
```

```
        /// <summary>
        /// Вычисляет градиент функции ошибки по матрице весов Embedding.
        /// </summary>
        public override Tensor[] Backward(Tensor gradOutput, Context context)
        {
            if (gradOutput == null) throw new
ArgumentNullException(nameof(gradOutput));

            Tensor weight = Inputs[0];
            Tensor gradWeight = null;

            if (weight.RequiresGrad)
            {
                // Запускаем Си-ядро обратного прохода с атомарным сложением на
GPU
                gradWeight = _nnClGlob.EmbeddingBackward(gradOutput,
_xTokensBuf, weight, _totalTokens);
            }

            // Возвращаем массив градиентов по количеству входов в Inputs (в
данном случае один элемент)
            return new[] { gradWeight };
        }
    }
}
```

NeuralNetwork

```
namespace cstorchcl.core
{
    /// <summary>
    /// Базовый класс для создания нейросетевых моделей (Аналог torch.nn.Module
в PyTorch).
    /// Объединяет глобальные операции, управляет параметрами и контекстом
вычислений.
    /// </summary>
    public abstract class NeuralNetwork : IDisposable
    {
        /// <summary>
        /// Диспетчер тяжелых глобальных ядер OpenCL.
        /// </summary>
        protected readonly NnClGlob NnClGlob;

        /// <summary>
        /// Контекст вычислений OpenCL.
        /// </summary>
        protected readonly ComputeContext ComputeContext;

        /// <summary>
        /// Список всех обучаемых тензоров-параметров модели (веса, смещения).
        /// </summary>
        private readonly List<Tensor> _parameters = new List<Tensor>();

        private bool _isDisposed = false;

        /// <summary>
        /// Инициализирует модель, привязывая её к контексту вычислений.
        /// </summary>
        protected NeuralNetwork(ComputeContext computeContext, NnClGlob
nnClGlob)
        {
```

```
            ComputeContext = computeContext ?? throw new
ArgumentNullException(nameof(computeContext));
            NnClGlob = nnClGlob ?? throw new
ArgumentNullException(nameof(nnClGlob));
        }

        /// <summary>
        /// Регистрирует тензор как обучаемый параметр модели.
        /// </summary>
        protected Tensor RegisterParameter(int[] shape, bool requiresGrad =
true)
        {
            // Параметры модели изначально создаются с флагом RequiresGrad =
true
            var param = new Tensor(shape, ComputeContext.Context,
NOpenCL.MemoryFlags.ReadWrite, requiresGrad);
            _parameters.Add(param);
            return param;
        }

        /// <summary>
        /// Возвращает список всех параметров модели для оптимизатора.
        /// </summary>
        public IEnumerable<Tensor> GetParameters() => _parameters;

        /// <summary>
        /// Обнуляет градиенты всех параметров перед новым шагом обучения.
        /// </summary>
        public void ZeroGrad()
        {
            var queue = DeviceQueueRegistry.GetCurrent();
            foreach (var param in _parameters)
            {
                if (param.Grad != null)
                {
                    // Создаем временный хост-массив нулей для очистки буфера
градиента
                    float[] zeroPattern = new float[param.Grad.Size];
                    for (int i = 0; i < zeroPattern.Length; i++)
                        zeroPattern[i] = 0;
                    long sizeInBytes = (long)param.Grad.Size * sizeof(float);
                    unsafe
                    {
                        fixed(float* zp = zeroPattern)
                        {
queue.EnqueueWriteBuffer(param.Grad.DataBuffer, true, 0, sizeInBytes,
(IntPtr)zp);
                        }
                    }

                }
            }
            queue.Finish();

            // Также очищаем хранилище промежуточных контекстов слоев LIFO
            GradientStorage.Clear();
        }

        /// <summary>
        /// Освобождает веса модели из VRAM видеокарты.
        /// </summary>
        public void Dispose()
        {
            if (_isDisposed) return;
```

```
            foreach (var param in _parameters)
            {
                param.Dispose();
            }
            _parameters.Clear();

            _isDisposed = true;
        }
    }
}
```

Мой вопрос сводиться к следующему : можно ли реализовать объект LSTM как 1 монолитное OpenCL ядро?

Если правильно понимаю, LSTM предстовляет из себя последовательность применения сигмойды к весам и если сделать так чтобы веса на всех шагах были одинаковы (тоесть прдставить их монолитным OpenCL float массивом N*T (где N размерность весов в слое ,а T количетво слоев) то можно используя opencl синхронизацию получсить неплохую производительбность за счет отсувия необходимости возвращать данные хосту на каждом шаге.

Единственно я не совмес понимаю , следует ли реализовывать lstm как объект глобального слоя (как bmm,softmax и т. д.) или же егоь нужно реализовывать как NeuroNetwork?(на подробие линейногос лоя).